Multi-layer perceptrons

Plan

1. **Representation and structure**
2. Learning: Backpropagation
3. Optimisation of learning

Neural networks I

- Logistic regression is the simplest neural network:
  - single layer
  - *model*
    - binary-class case: $P'(C_1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$, single hidden unit, learn $\mathbf{w}$.
    - K classes case: $P'(C_i|\mathbf{x}) = \frac{\exp \mathbf{w}_i^T\mathbf{x}}{\sum_{j=1}^{K} \exp \mathbf{w}_j^T\mathbf{x}}$, K hidden units, learn $\{\mathbf{w}_i | i = 1...k\}$.
  - *error function*, Cross-entropy:

$$L(\mathbf{y}, \mathbf{x}) = -\sum_{j=1}^{K} P(C_j|\mathbf{x}) \ln P'(C_j|\mathbf{x}) = -\sum_{j=1}^{K} y_j \ln P'(C_j|\mathbf{x})$$

$\mathbf{y}$ is the one-hot vector encoding of the class label.

Neural networks II

- a less trivial NN is a repetitive application of logistic regression with the error function at the last layer.
- at the heart of NNs we have function composition
- To do optimization over function composition we make heavy use of the *chain rule*
- The chain rule comes under the name *backpropagation* in neural networks.

Chain rule, reminder, I

- let:

$$\begin{aligned} f(\mathbf{x}) &: \mathbf{x} \rightarrow y, \mathbf{x} \in R^d, y \in R \\ g(y) &: y \rightarrow z, y \in R, z \in R \end{aligned}$$

- The partial derivative of the composition $z = g(f(\mathbf{x}))$ with the ith dimension of $\mathbf{x}$ is given by:

$$\frac{\partial z}{\partial x_i}$$

Chain rule, reminder, I

- let:

$$
\begin{aligned}
f(\mathbf{x}) &: \mathbf{x} \rightarrow y, \mathbf{x} \in R^d, y \in R \\
g(y) &: y \rightarrow z, y \in R, z \in R
\end{aligned}
$$

- The partial derivative of the composition $z = g(f(\mathbf{x}))$ with the ith dimension of $\mathbf{x}$ is given by:

$$
\begin{aligned}
\frac{\partial z}{\partial x_i} &= \frac{\partial z}{\partial y}\frac{\partial y}{\partial x_i} \\
&= \frac{\partial g(y)}{\partial y}\frac{\partial f(\mathbf{x})}{\partial x_i}
\end{aligned}
$$

Chain rule, reminder, II

- what happens when y is a vector? i.e. let

$$\begin{aligned} f(\mathbf{x}) &: \mathbf{x} \rightarrow \mathbf{y}, \mathbf{x} \in R^d, \mathbf{y} \in R^n \\ g(\mathbf{y}) &: \mathbf{y} \rightarrow z, \mathbf{y} \in R^n, z \in R \end{aligned}$$

- What is the meaning of $\frac{\partial z}{\partial x_i}$?
  - describes the sensitivity of z to x_i
- In how many ways does x_i affect z?
  - in n ways. Which ones?

Chain rule, reminder, II

- what happens when y is a vector? i.e. let

$$\begin{aligned} f(\mathbf{x}) &: \mathbf{x} \rightarrow \mathbf{y}, \mathbf{x} \in R^d, \mathbf{y} \in R^n \\ g(\mathbf{y}) &: \mathbf{y} \rightarrow z, \mathbf{y} \in R^n, z \in R \end{aligned}$$

- What is the meaning of $\frac{\partial z}{\partial x_i}$?
  - describes the sensitivity of z to x_i
- In how many ways does x_i affect z?
  - in n ways. Which ones?
  - all $y_j, i = j...n$
- Thus to establish the sensitivity/partial derivative of z with x_i we need all of them:

$$\frac{\partial z}{\partial x_i} =$$

Chain rule, reminder, II

- what happens when y is a vector? i.e. let

$$\begin{aligned} f(\mathbf{x}) &: \mathbf{x} \rightarrow \mathbf{y}, \mathbf{x} \in R^d, \mathbf{y} \in R^n \\ g(\mathbf{y}) &: \mathbf{y} \rightarrow z, \mathbf{y} \in R^n, z \in R \end{aligned}$$

- What is the meaning of $\frac{\partial z}{\partial x_i}$?
  - describes the sensitivity of z to x_i
- In how many ways does x_i affect z?
  - in n ways. Which ones?
  - all $y_j, i = j...n$
- Thus to establish the sensitivity/partial derivative of z with x_i we need all of them:

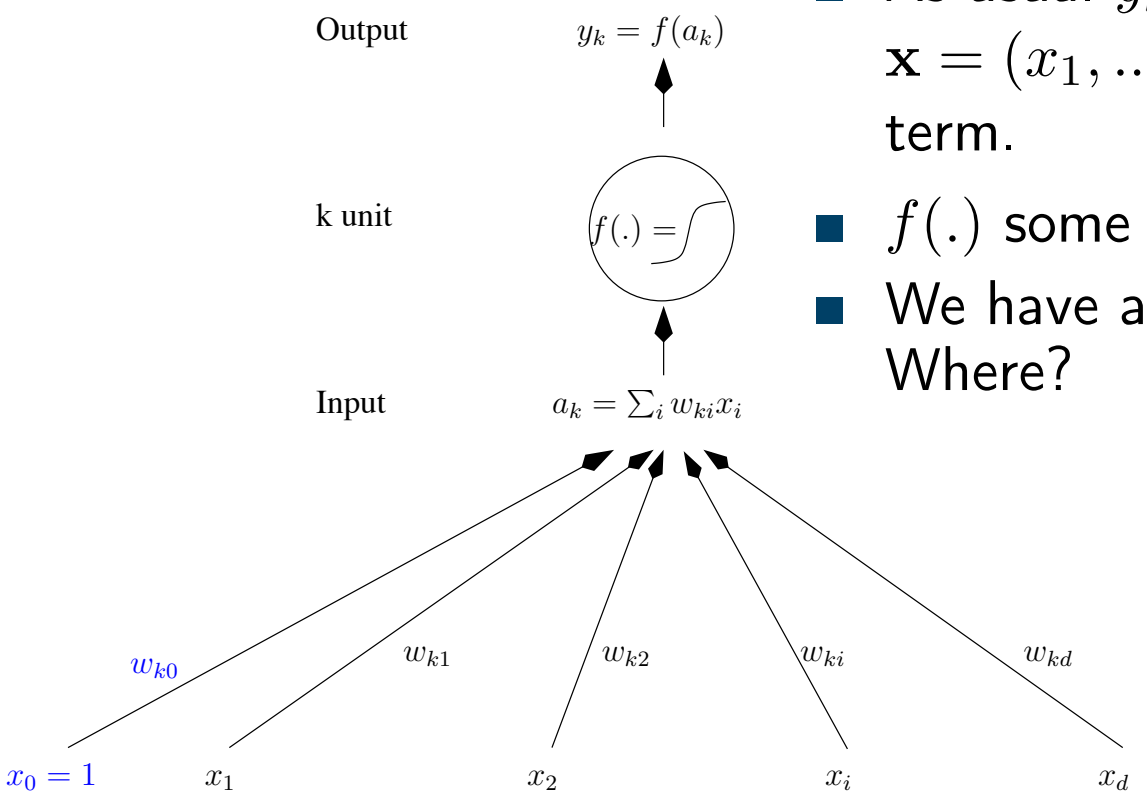
$$\frac{\partial z}{\partial x_i} = \frac{\partial z}{\partial y_1}\frac{\partial y_1}{\partial x_i} + \frac{\partial z}{\partial y_2}\frac{\partial y_2}{\partial x_i} + ... + \frac{\partial z}{\partial y_n}\frac{\partial y_n}{\partial x_i}$$

the *multivariate chain rule*

NNs very short summary

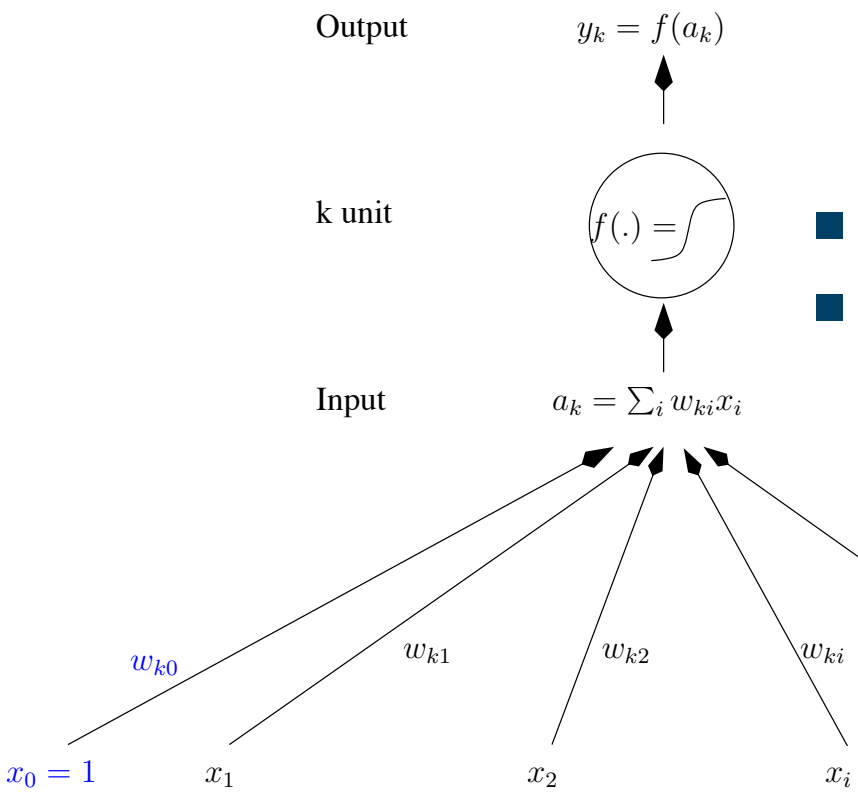
- standard NNs can be thought as an hierarchical composition of logistic regression models, loss function at the last layer.
- To compute the gradient of the loss
  - we rely on the chain rule to address the composition and the fact that indermediate results will be vectors
- NN course could stop here.

Single Layer Networks



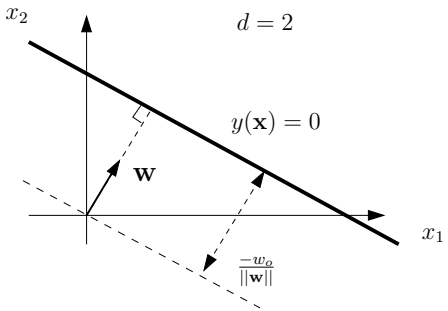
- As usual y_k the kth output, $\mathbf{x} = (x_1, ..., x_d)^T$ the input, w_{k0} the bias term.
- $f(.)$ some transformation function
- We have already seen such structures. Where?

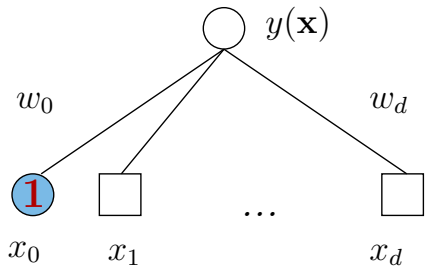
Single Layer Networks



- As usual y_k the kth output, $\mathbf{x} = (x_1, ..., x_d)^T$ the input, w_{k0} the bias term.
- $f(.)$ some transformation function
- We have already seen such structures. Where?
  - Perceptron: $f(a) = sign(a), k = 1$
  - Logistic regression :
    - $f(a) = \frac{1}{1+e^{-a}}, k = 1$
    - $f(a_k) = \frac{\exp a_k}{\sum_k \exp a_k}, k \in [1...c]$
  - Fisher's linear discriminant $f(.) = I(.)$

Limitations of perceptron



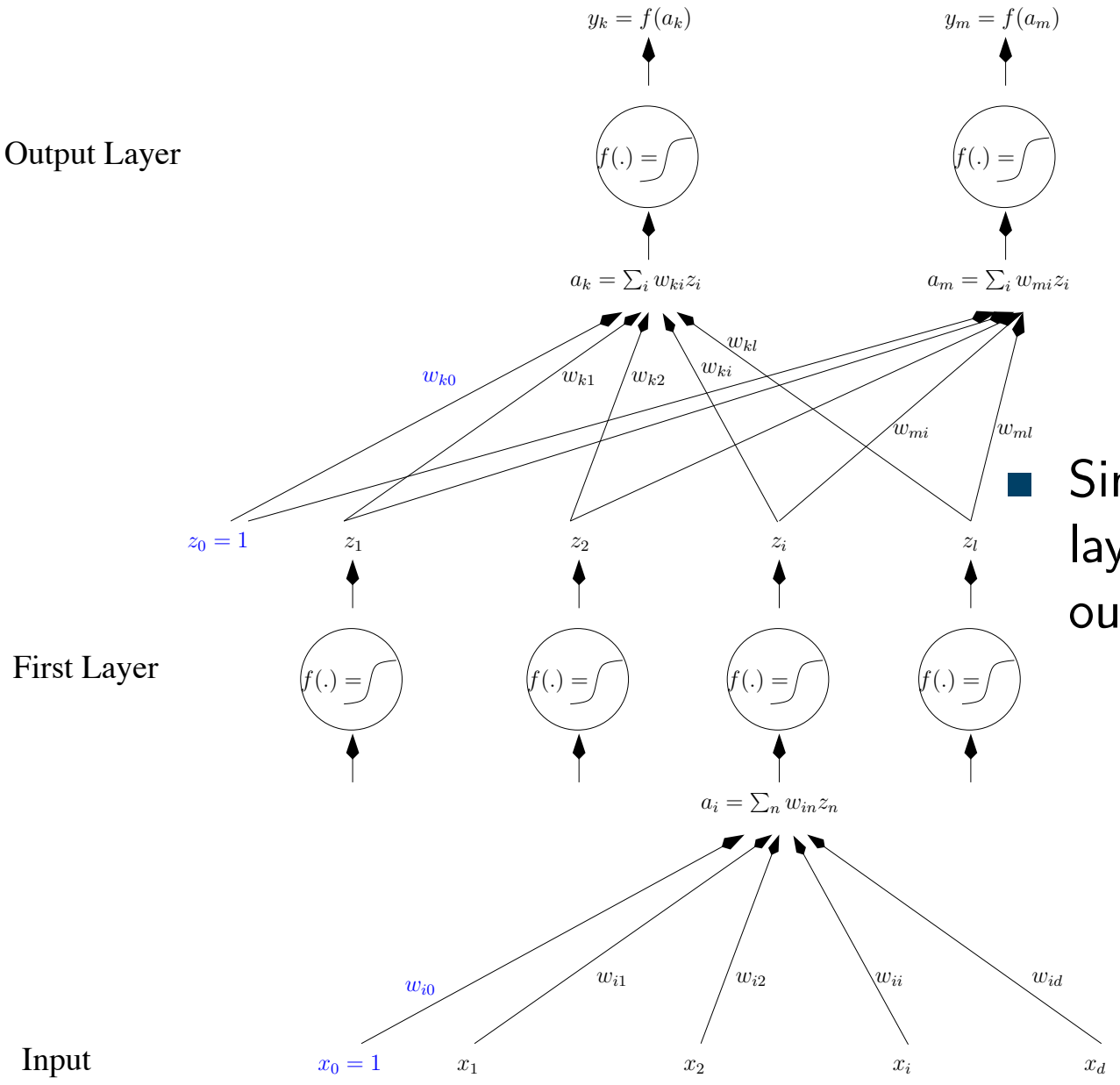


- It can only work on linearly seperable problems
- Most problems are not linearly seperable
- To get more complex surfaces we can supperimpose many nodes that use *non-linear* activation functions
- One layer: a set of connections/weights followed by the respective nodes.

MLP structure I

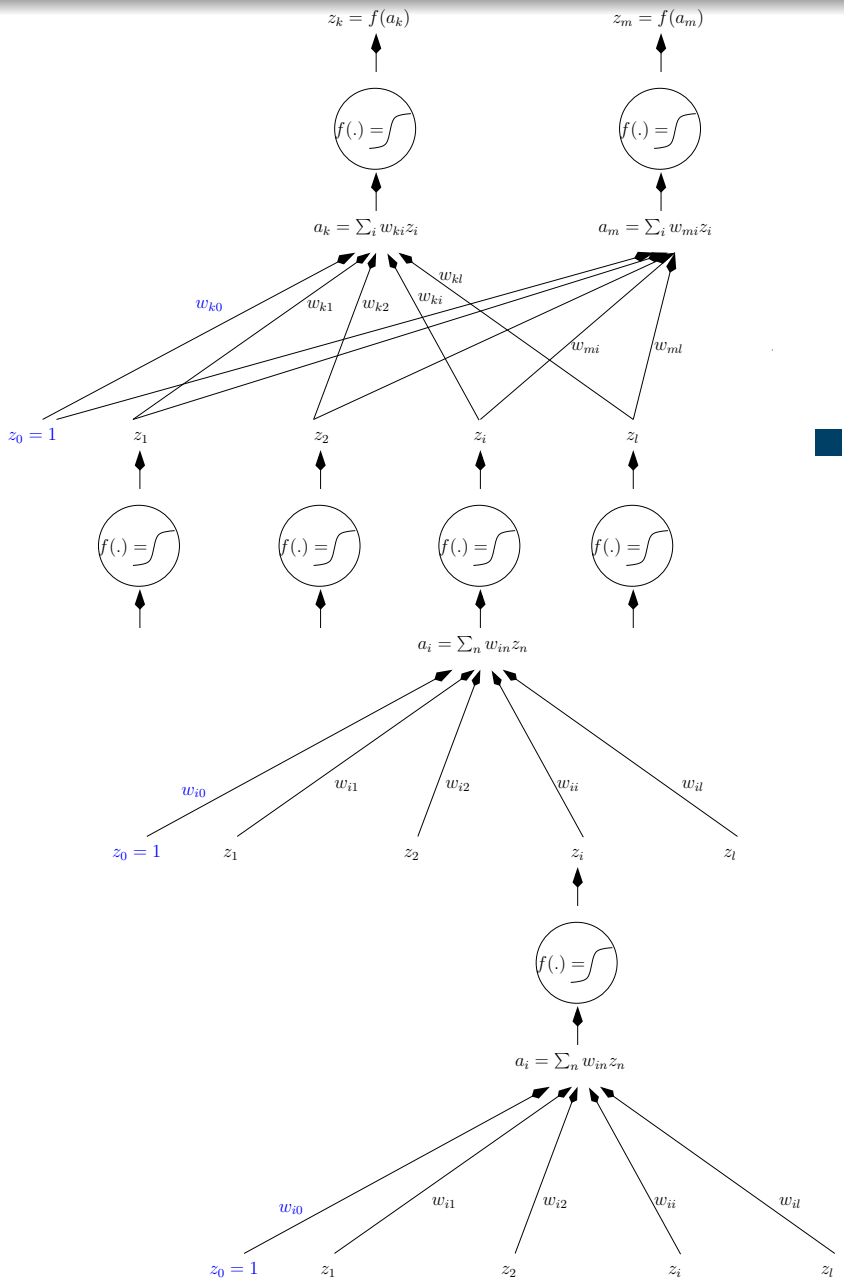
- Multi-Layer Perceptrons (MLP) = networks of many layers
- acyclic (feedforward): no backward connections
- completely connected: each node of a layer is connected to all nodes of the previous layer

MLP structure II



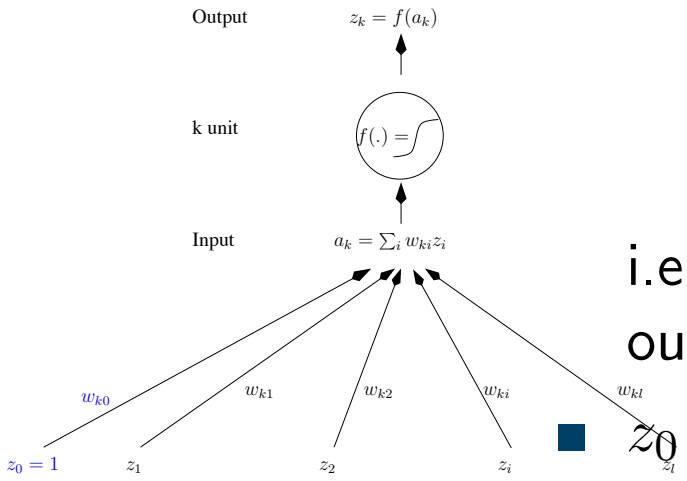
- Simplest example : 2 layers one hidden, one output

MLP structure III



- Nowdays five-layer, and more, nets are common: *deep neural networks* and *deep learning*. For example:
  - [2] use eight layers with 60 million parameters.
  - deep face, [3], 120M parameters, and [1], 10B parameters.

The hidden unit



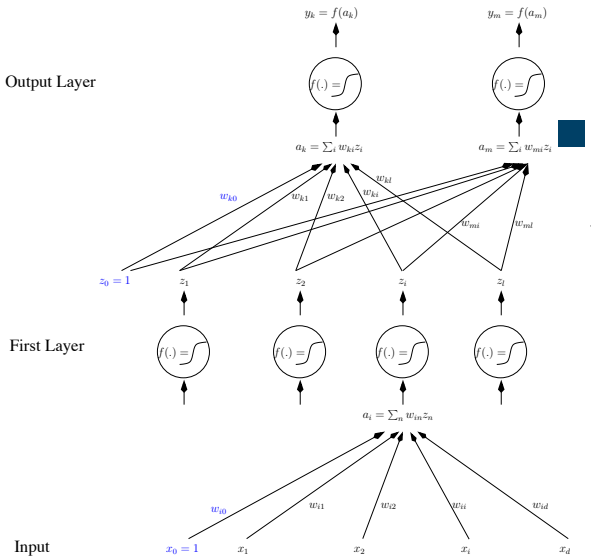
- Each unit k in some layer receives an activation value a_k given by:

$$a_k = \sum_i^l w_{ki} z_i = \mathbf{w}_k^T \mathbf{z}$$

i.e. a_k is a weighted combination of the outputs of the l units of the previous layer.

- $z_0 = 1$ and w_{k0} are there to incorporate the bias in the model.
- The output z_k of the unit is a non-linear function $f(.)$ of the activation value

The output layer



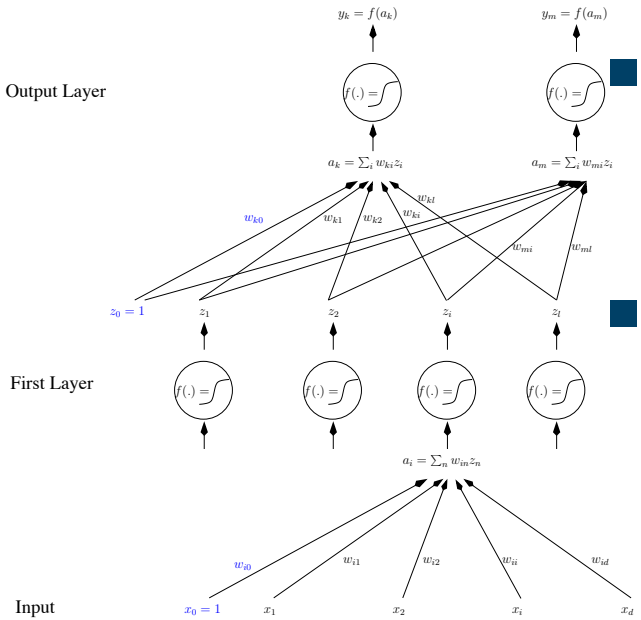
- Number of output units:
  - Regression: single unit corresponding to the target value
  - Binary classification: single unit corresponding to the "positive" class
  - Multi-class classification: one unit/class
- Similar to the hidden layers each k output unit gets as activation the linear combination of the outputs of the units of the previous layer:

$$a_k = \sum_i^l w_{ki} z_i$$

- and produces as output the non-linear transformation:

$$y_k = f(a_k)$$

Analytical function of a two layer MLP



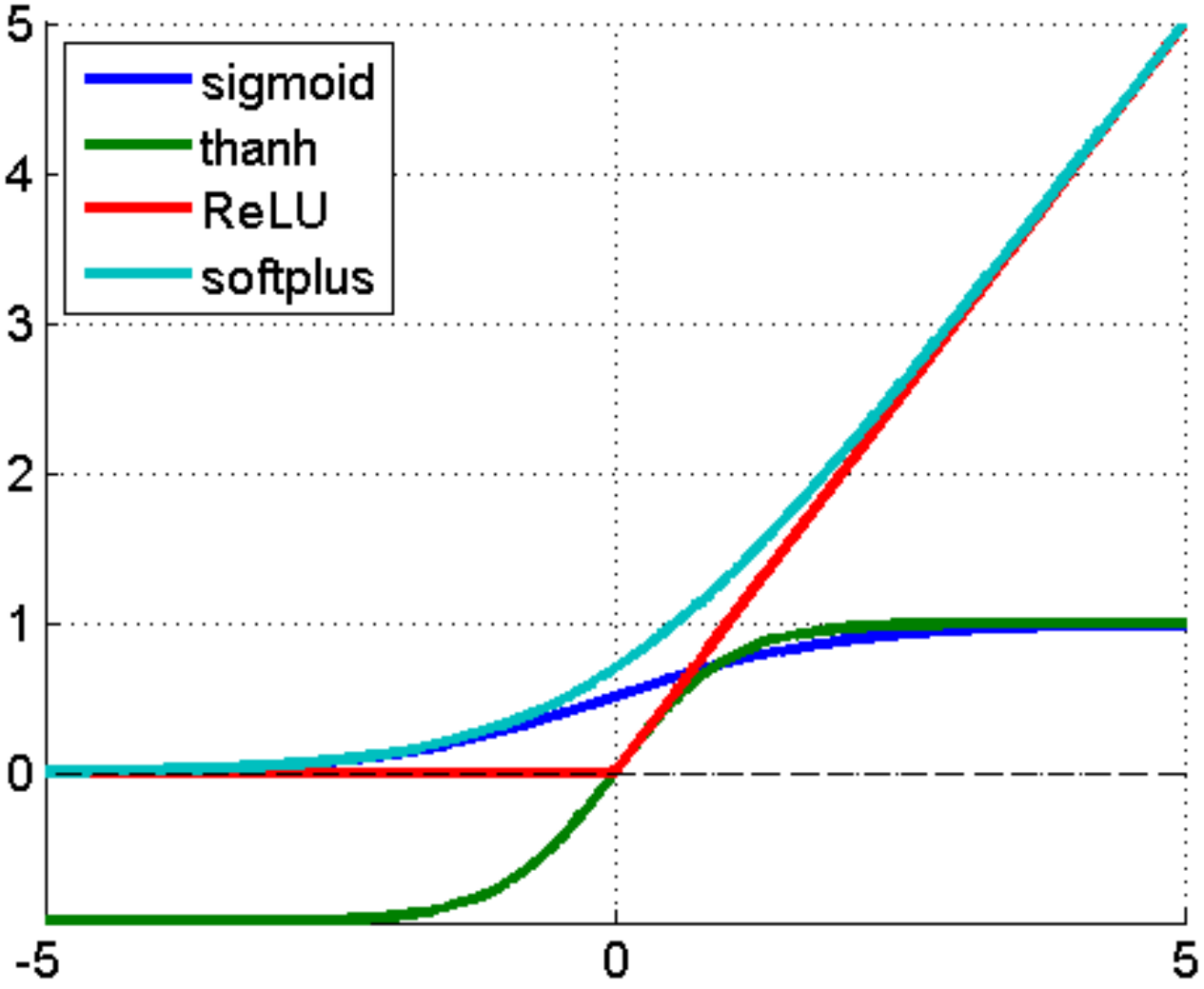
- An acyclic network can be seen as a composition of functional transformations of its inputs
- For example a two-layer MLP is:

$$y_k = g\left(\sum_{i=0}^{l} w_{ki}^{(2)} f\left(\sum_{n=0}^{d} w_{in}^{(1)} x_n\right)\right)$$

Activation functions I

- We need a *differentiable* function so that we can do gradient descent
- the activation function is typically *non-linear*
  - sigmoid $\sigma : \mathbb{R} \to [0,1]$: $f(a) = \frac{1}{1+e^{-a}}$
  - tanh: $\mathbb{R} \to [-1,1]$: $f(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$
  - Rectified linear units (ReLU): $f(a) = \max(0, a)$
    - currently the most popular non-linearity,
    - introduces sparsity, no exploding or vanishing derivatives problem
  - soft plus : $f(a) = \ln(1 + e^x)$ smooth approximation of ReLU
- At the output layer the activation function can be:
  - regression : linear (identity)
  - classification: sigmoid or softmax

Activation functions II



Plan

1. Representation and structure
2. **Learning: Backpropagation**
3. Learning Optimisation

Error optimization in MLPs

- As always we are given some error function $E(\mathbf{w})$
- which we would like to optimise (minimise) with respect to the parameters of the model, here the vector of weights $\mathbf{w}$:

$$\mathbf{w} \equiv [w_{ij} | \text{weight of the connection of the } j \text{ unit to the } i \text{ unit}]$$

- The most classical way to do so for MLPs is by gradient descent

$$\begin{aligned} \nabla E[\mathbf{w}] &\equiv [\frac{\partial E}{\partial w_{10}}, \frac{\partial E}{\partial w_{11}}, \ldots, \frac{\partial E}{\partial w_{ij}}, \ldots \frac{\partial E}{\partial w_{ml}}] \\ \mathbf{w}^{(\tau+\mathbf{1})} &\leftarrow \mathbf{w}^{(\tau)} - \eta \nabla E[\mathbf{w}^{(\tau)}] \end{aligned}$$

η the learning rate

How to compute the gradient I

- We will make heavy use of the chain rule.
- The contribution of a weight w_{ij} to the error E comes only through:
  - the activation level $a_i = \sum_k w_{ik} z_k$
  - of the ith unit to which it connects the output z_j of the jth unit; Thus:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial a_i} \frac{\partial a_i}{\partial w_{ij}} \tag{1}$$

- We have $a_i = \sum_k w_{ik} z_k \Rightarrow \frac{\partial a_i}{\partial w_{ij}} = z_j$
- And we set $\delta_i \equiv \frac{\partial E}{\partial a_i}$ (δ_i is called the error at the i node)
- Thus w_{ij} gradient can be written as:

$$\frac{\partial E}{\partial w_{ij}} = \delta_i z_j \tag{2}$$

How to compute the gradient II

- So the gradient of the error with respect to the w_{ij} weight that connects the ouput of unit j to the input of unit i is given according to equation:

$$\frac{\partial E}{\partial w_{ij}} = \delta_i z_j$$

  by the product of the output z_j of the unit j with the error δ_i of the i input unit.
- *Thus* to get the gradient we need to compute δ_i at each *hidden* and *output* unit.

Compute δ_k at the kth output unit

- We have:

$$
\begin{aligned}
\delta_k &= \frac{\partial E}{\partial a_k} \\
y_k &= f(a_k) \\
&\Rightarrow \\
\delta_k &= \frac{\partial E}{\partial y_k}\frac{\partial y_k}{\partial a_k} = \frac{\partial E}{\partial y_k}\frac{\partial f(a_k)}{\partial a_k} \\
&= \frac{\partial E}{\partial y_k} f'(a_k) \qquad (3)
\end{aligned}
$$

- thus we just need
  - the derivative of the error with the y_k output
  - and the derivative of the activation function evaluated at the input a_k of the k output unit.

Compute δ_j at the jth hidden unit

- We have: $\delta_j = \frac{\partial E}{\partial a_j}$
- a_j affects the error through the activation levels a_k of every unit k to which the output of the j unit is passed as input. Thus:

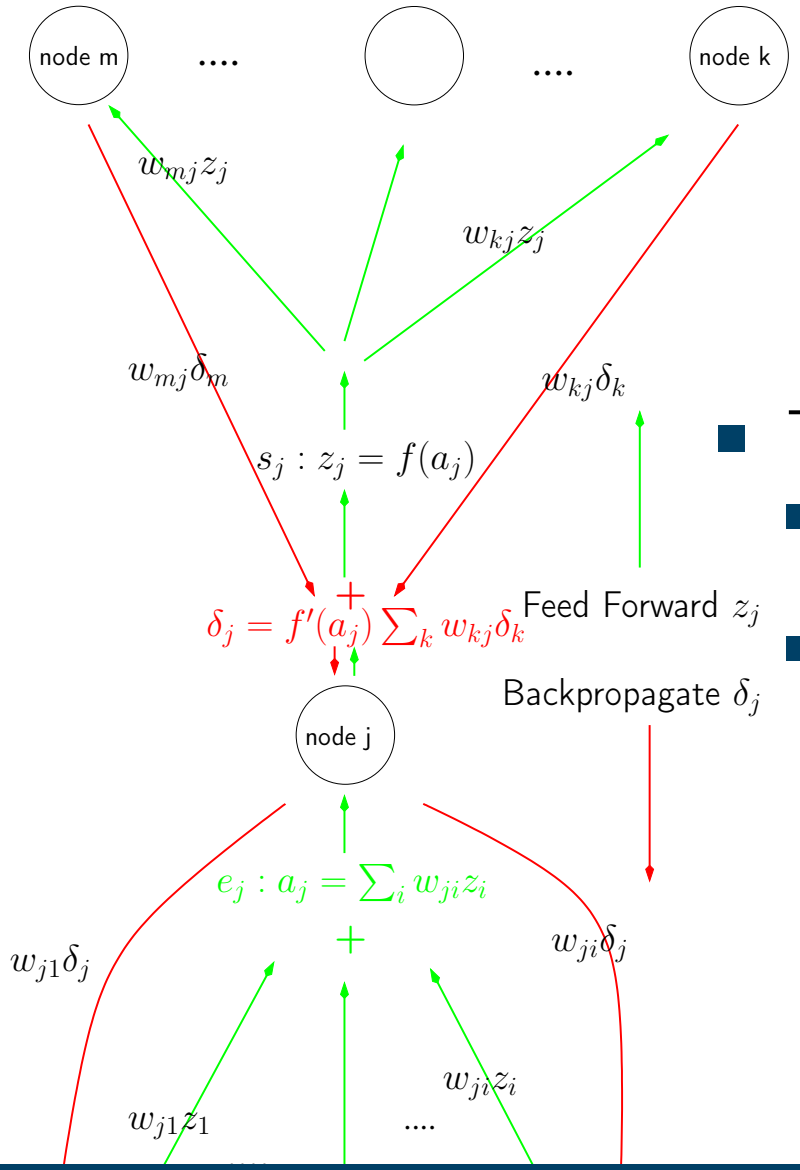
$$\frac{\partial E}{\partial a_j} = \sum_k \frac{\partial E}{\partial a_k}\frac{\partial a_k}{\partial a_j} = \sum_k \delta_k \frac{\partial a_k}{\partial a_j} \tag{4}$$

- Since: $a_k = \sum_i w_{ki} z_i = \sum_i w_{ki} f(a_i) \Rightarrow \frac{\partial a_k}{\partial a_j} = w_{kj} f'(a_j)$
- giving eventually that:

$$\delta_j = \frac{\partial E}{\partial a_j} = f'(a_j) \sum_k \delta_k w_{kj} \tag{5}$$

or in words: the error δ_j of a node j is the weighted sum of the errors of the nodes to which it passes its output.

Demonstration of MLP info-flow around the j node



- Training in MLPs goes in two steps:
  - Forward pass: inputs are passed forward in the net computing the a_ks and the z_ks
  - Backward pass: errors, δ_is, are passed backwards allowing for the computation of the gradient for every weight.

Backpropagation (on-line)

Initialise the weights
repeat
  for all $\mathbf{x}_n$ **do**
    Forward pass $\mathbf{x}_n$ and calculate the activations of the hidden units and the outputs.
    {Compute the δ_k of the output units (eq 3)}
    for all k output units **do**
      compute δ_k; $\Delta w_{kj} \leftarrow -\eta \delta_k z_j$
    end for
    {Back-propagate to compute the δ_j of the hidden units (eq 5)}
    for all j hidden units **do**
      Compute δ_j; $\Delta w_{ji} \leftarrow -\eta \delta_j z_i$
    end for
    {Update the weights }
    for all w_{ji} **do**
      $w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$
    end for
  end for{for all $\mathbf{x}_n$}
until convergence

Backpropagation (batch)

Initialise the weights
repeat
  $\Delta w_{kj} \leftarrow 0$, $\Delta w_{ji} \leftarrow 0$
  for all $\mathbf{x}_n$ **do**
    Forward pass $\mathbf{x}_n$ and calculate the activations of the hidden units and the outputs.
    {Compute the δ_k of the output units (eq 3)}
    for all k output units **do**
      compute δ_k; $\Delta w_{kj} \leftarrow \Delta w_{kj} - \eta \delta_k z_j$
    end for
    {Back-propagate to compute the δ_j of the hidden units (eq 5)}
    for all j hidden units **do**
      Compute δ_j; $\Delta w_{ji} \leftarrow \Delta w_{ji} - \eta \delta_j z_i$
    end for
  end for{for all $\mathbf{x}_n$}
  {Update the weights}
  for all w_{ji} **do**
    $w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$
  end for
until convergence

Online and batch backpropagation

- online:
  - We compute errors/derivatives and we adjust weights after seeing each example
  - Sensitive to the order of presentation of examples
  - Converges faster in the beggining but slower in general
  - The method of choise for very large datasets (stochastic gradient descent)
- Batch
  - We accumulate the derivative changes over the examples
  - We ajust the weights at the end of each cycle (epoch).

Plan

1. Representation and structure
2. Learning: backpropagation
3. **Optimisation and learning**

Initialisation of weights

- Objective : converge fast
- Initialise weights to random values
  - Uniformly distributed around 0 for the hiddent unitis
    $\rightarrow$ avoid extreme values on the sigmoid outputs
  - larger [-1,1] for the input units
    $\rightarrow$ if the weights w_{kj} are very smalle then δ_j and updates very small in the beggining
  - Initial weights should be different from one node to the other
    $\rightarrow$ if not all hidden units will do the same job

Learning rate η

Same discussion as in any gradient descent

- Value of η to be chosen experimentaly
- Very small: slow convergence
- Very large : oscilations, problematic convergences
- Rule of thumb : start with η large, and gradually reduce

Complexity of MLP backpropagation

- Computational complexity is

$$O(W)$$

where W = the number of parameters (weights and biases)

- Model complexity of a two layer MLP:

$$W = (d+1)h + (h+1)c$$

Model complexity control, MLP I

Reducing the number of hidden units

- Explore different numbers of hidden units and layers and choose the configuration that minimises the validation error
- Start with small numbers and increase during learning
- Start with a large network and prune
- Train a large number of small networks and combine

Model complexity control, MLP II

Regularisation : add a model complexity penalty

$$\tilde{E} = E + \Omega$$

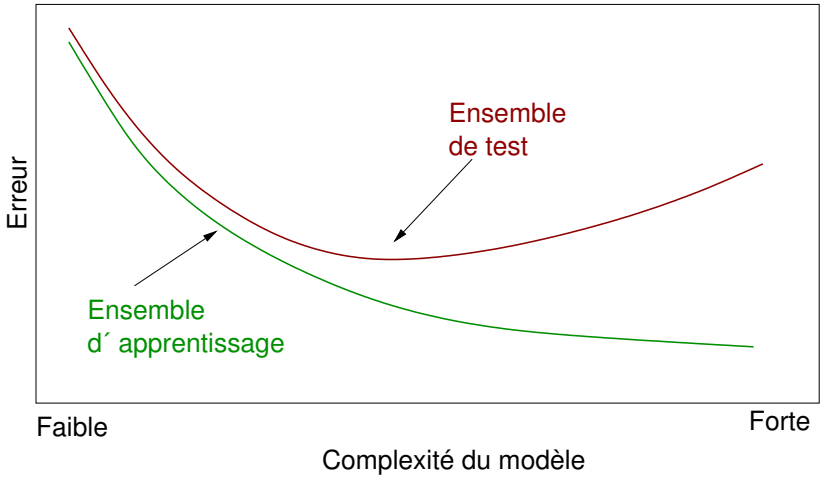
Ex. Weight decay

$$\Omega = \frac{1}{2} \sum_i w_i^2$$

To minimize $\tilde{E}$, we need to find the right trade-off between error (E) and model complexity Ω.

Model complexity control, MLP III

Early stopping : do not wait for convergence



- Fix the maximum number of epochs
- Stop if the training error does not decrease after k successive epochs
- Use a validation set, if validation error starts increasing stop.

MLP: summary

- can be used for classification and regression
- universal approximators
- robust to noisy
- we can generate arbitrary complex models
- training can be long
- regression and classification is fast
- have fallen to oblivion from the beggining of 90's
- back in fashion since 5 to 10 years now, known as deep neural networks (everybody is on them)
- big data, computational power and a number of tricks are responsible for the renaissance.

Books

- C. Bishop (2006). *Pattern Classification and Machine Learning*, Chapter 5: Neural Networks.
- C. Bishop (1995). *Neural Networks and Pattern Classification*, Chapter 4: The Multi-Layer Perceptron.
- Ian Goodfellow and Yoshua Bengio and Aaron Courville (2016). *Deep Learning*.

References I

Adam Coates, Brody Huval, Tao Wang, David J. Wu, Bryan C. Catanzaro, and Andrew Y. Ng.
Deep learning with COTS HPC systems.
In *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013*, volume 28 of *JMLR Proceedings*, pages 1337–1345. JMLR.org, 2013.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton.
Imagenet classification with deep convolutional neural networks.
In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 1097–1105. Curran Associates, Inc., 2012.

References II

Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato, and Lior Wolf.
Deepface: Closing the gap to human-level performance in face verification.
In *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*, pages 1701–1708. IEEE Computer Society, 2014.